

Phase-Adaptive Quantization: Design Space Exploration of 4-Bit MMA Rescale Pipelines

Yichong Zhang
yichongz@mit.edu

Wenye Xiong
wenyexio@mit.edu

I. PROJECT OVERVIEW

This project is based on **Example Project 4 (Quantization)**.

A. Problem and Motivation

Modern AI workloads consists of two fundamentally different computational phases. **Prefill** processes the entire input prompt in a single forward pass with large batch dimension ($M \gg 1$), making it **compute-bound**: the energy and latency bottleneck lies in arithmetic operations, particularly the multiply-accumulate datapath. **Decode** generates tokens autoregressively with $M = 1$, making it **memory-bandwidth-bound**: the bottleneck is streaming the full weight matrix from HBM for every generated token. These two phases have radically different hardware utilization profiles—yet current 4-bit quantization schemes apply a single, fixed pipeline to both.

All major 4-bit formats—NVIDIA’s NVFP4 (Blackwell), the Microscaling standard (MXFP4), and INT4 (GPTQ/AWQ)—are **bundled, one-size-fits-all designs**. Each implicitly fixes the entire MMA datapath: number of rescale levels, per-level granularity, scale-factor precision, and accumulator precision. On their native platforms these schemes are highly optimized: NVIDIA’s Tensor Cores implement NVFP4 with deep microarchitectural tuning, achieving excellent throughput on Blackwell GPUs. However, the format-level design choices—FP32 rescale, two-level scaling, fixed block size—are conservative because GPU vendors must guarantee numerical safety across arbitrary workloads and both inference phases.

For **domain-specific AI accelerator design**, this one-size-fits-all constraint does not apply. When designing a custom accelerator for a known deployment scenario—a particular model family, a target prefill/decode ratio, a specific latency budget—the designer has the freedom to co-optimize the quantization format with the hardware datapath. Blindly adopting NVFP4 or MXFP4 as-is means inheriting conservatism that is unnecessary for the target workload. This is precisely the design scenario our framework addresses: **given a target workload, what is the optimal quantization pipeline to build into the hardware?** The answer, as we show, depends on the inference phase:

- **During prefill**, the rescale pipeline dominates energy. Our AccelForge modeling of the NVFP4 W4A4 pipeline on Llama-style FFN GEMMs ($M=128$) shows that **rescale stages account for 74% of total energy** (Figure 1), and the full NVFP4 pipeline costs **$1.27\times$ the energy of an FP16 baseline**—quantization actually *increases* energy during prefill despite the latency improvement. Cheaper rescale formats would directly address this bottleneck.
- **During decode** ($M=1$), memory bandwidth dominates. Here NVFP4 W4A4 achieves **$0.90\times$ energy and $0.59\times$ latency** vs. FP16—both metrics improve, because 4-bit weights cut data movement in half. But the rescale overhead is still 64.5% of W4A4 energy; compared to an ideal zero-overhead 4-bit multiply, the full pipeline costs $2.8\times$ in energy.

These preliminary results reveal a striking asymmetry: **the same NVFP4 pipeline is energy-beneficial for decode but energy-harmful for prefill**. This directly motivates phase-adaptive quantization—using cheaper rescale for compute-bound prefill, while retaining the full pipeline for bandwidth-bound decode where the data-movement savings outweigh the rescale cost.

Recent works have made isolated improvements along individual pipeline dimensions—HiFloat4 [1] proposes a fixed three-level hierarchy, AXE [2] treats accumulator precision as tunable, MixPE [3] explores dequantization placement, and “Is Finer Better?” [4] analyzes block-size/scale-format interactions. However, each produces a single new hand-designed point in the space, and **none considers the prefill/decode dichotomy**. The problem has never been formulated as a joint DSE that decouples the two phases and searches for phase-specific optima.

B. Key Questions

Core question: When designing an AI inference accelerator, can we define a structured search space over the 4-bit MMA rescale pipeline—decomposed into orthogonal dimensions (rescale levels, block granularity, scale format, accumulator precision)—and demonstrate that prefill and decode have *different* Pareto-optimal configurations within this space, thereby justifying phase-adaptive quantization over adopting any single existing scheme as-is?

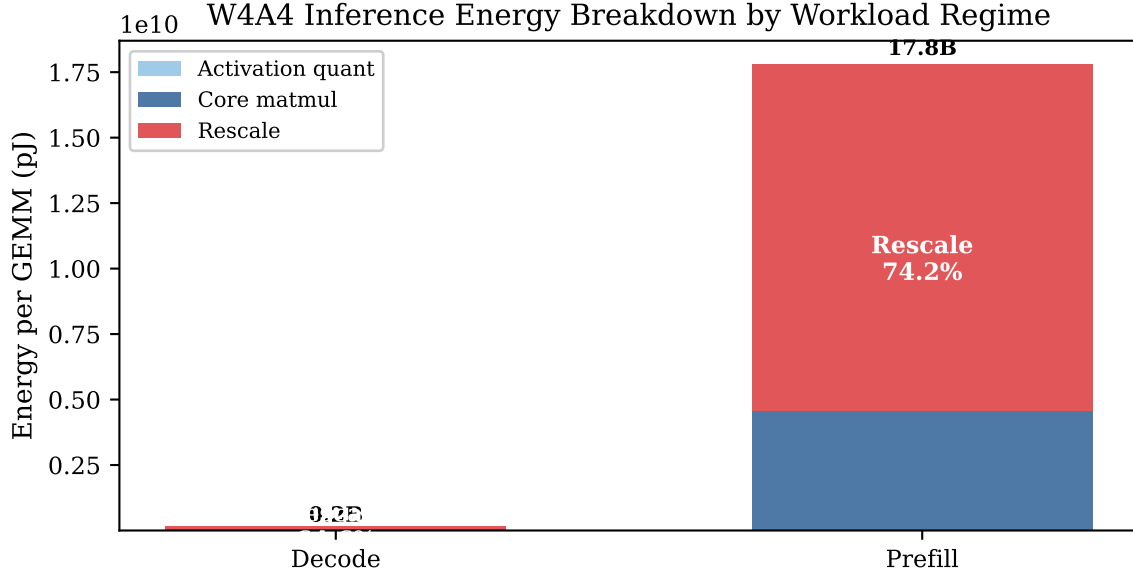


Fig. 1. NVFP4 W4A4 energy breakdown for Llama-style FFN GEMMs. Rescale operations dominate in both phases: 74% of energy during prefill ($M=128$), 64.5% during decode ($M=1$). The core FP4 MatMul accounts for only 26–35%.

Our contribution is not a single better configuration, but **the search space definition and the phase-decoupling methodology** for custom accelerator design. The finding that prefill and decode prefer different configurations is the central validation—it proves that blindly adopting a general-purpose format like NVFP4 into a custom datapath leaves efficiency on the table, and that co-optimizing the quantization pipeline with the target deployment scenario is valuable.

C. Hypotheses

- 1) **Prefill favors cheaper rescale.** In the compute-bound regime (large M), the rescale pipeline is the dominant energy contributor. Lower-precision rescale (FP16, bit-shift) can achieve comparable accuracy at significantly lower energy.
- 2) **Decode favors more aggressive quantization.** In the memory-bound regime ($M = 1$), configurations with coarser granularity or fewer rescale levels may be Pareto-optimal because bandwidth savings dominate.
- 3) **The optimal configuration differs across both phases and workloads.** LLM, VLM, and VLA workloads have distinct GEMM shape profiles (N , K dimensions), leading to different compute-vs-memory balance and thus different optimal rescale pipelines. The Pareto frontier for each (workload, phase) pair occupies a different region of the accuracy–energy plane. This two-dimensional workload-dependence—across phases *and* application domains—is the key validation of our framework.
- 4) We target $\geq 2\times$ **energy reduction** in the rescale/accumulation stages relative to NVFP4 for the workload-optimal configuration, with configurations on the Pareto frontier **dominating NVFP4 and MXFP4** in all evaluated scenarios.

II. TECHNICAL CONTRIBUTIONS

A. Current Solutions and Related Work

Current 4-bit schemes each bundle a fixed rescale pipeline, applied uniformly to both prefill and decode (Table I).

TABLE I
EXISTING 4-BIT SCHEMES AS FIXED PIPELINE CONFIGURATIONS.

Scheme	# Levels	Scale Fmt	Granularity	Rescale Op
INT4-GPTQ	1	FP16+zp	g=128	FP16 dequant
MXFP4	1	E8M0	b=32	Bit-shift
NVFP4	2	FP8+tensor	b=16+tensor	FP32 $\times 2$
HiFloat4	3	E6M2+E1+E1	hierarchical	Mixed

It is important to distinguish between two categories of quantization work. **Format-level schemes** (NVFP4, MXFP4, HiFloat4, MX+, M²XFP) define a fixed data layout and a simple, regular arithmetic pipeline—block scaling, multiply, accumulate,

rescale—that maps directly to dedicated hardware: shifters, fixed-precision multipliers, and accumulators. Each configuration fully specifies a *synthesizable datapath*. In contrast, **algorithm-level methods** (SmoothQuant, GPTQ/AWQ, QuIP) involve complex runtime logic—per-channel activation migration, Hessian-based weight rounding, incoherence processing—that is inherently sequential and data-dependent, making them unsuitable for dedicated hardware pipelines. Our search space operates exclusively at the **format/datapath level**: every point in the space corresponds to a concrete, hardware-realizable MMA pipeline with no complex control logic, just as NVFP4 and MXFP4 do today.

Among format-level works, all recent proposals share a common limitation: **each produces a single fixed design applied uniformly to all inference phases**. HiFloat4 fixes its three-level hierarchy as a single format; AXE optimizes accumulator precision in a software-only PTQ framework without hardware cost modeling; MixPE performs Pareto DSE over dequantization *placement* but does not touch the internal rescale pipeline structure; M²XFP [5] and MX+ [6] enhance MX through metadata augmentation, each producing one new fixed scheme. None considers that prefill and decode have fundamentally different compute-vs-memory profiles and may benefit from different pipeline configurations.

What’s missing: (1) A structured DSE formulation decomposing the rescale pipeline into orthogonal dimensions. (2) Phase-aware evaluation that separately models hardware cost under prefill and decode regimes. (3) The insight that the optimal configuration is phase-dependent, making phase-adaptive quantization both feasible and necessary.

B. Our Approach: Phase-Adaptive Quantization via Search Space Decomposition

We decompose the 4-bit MMA datapath into independently tunable dimensions, define a structured search space, and evaluate each configuration under **both prefill and decode regimes** to identify phase-specific optima. Critically, every configuration in our search space specifies a **hardware-friendly, regular datapath**—block scaling followed by multiply-accumulate followed by rescale—with no data-dependent branching or complex control logic. Each point can be directly implemented as a fixed-function pipeline in silicon, just as NVFP4 and MXFP4 are today. We fix weight format (E2M1) and activation format (FP16/FP8) and focus on three core dimensions:

Dimension 1: Rescale pipeline structure—the number of rescale levels (0, 1, or 2) and block granularity at each level ($b \in \{16, 32, 64, \text{per-channel}\}$; for 2-level, coarse $B \in \{256, \text{per-channel, per-tensor}\}$). Finer granularity and more levels improve accuracy but add rescale operations and scale-factor storage.

Dimension 2: Scale-factor format (per level, independently)—FP32, FP16, BF16, FP8 E4M3, E8M0 (power-of-2 bit-shift, nearly free), or fixed-point. In a 2-level pipeline the fine and coarse levels may use different formats, e.g., E8M0 shift at fine level + FP16 at coarse level.

Dimension 3: Accumulator precision—FP32, FP16, BF16, INT32, or segmented (accumulate N partial products in FP16, then reduce to FP32).

After pruning invalid combinations (e.g., accumulator precision $\geq$ operand precision), we estimate ~ 50 viable configurations.

1) *Locating existing schemes:* Every existing scheme is a single hand-designed point in this space:

- **NVFP4** = {2-level ($b=16$, $B=\text{tensor}$), FP32 rescale $\times 2$, FP32 acc}
- **MXFP4** = {1-level ($b=32$), E8M0 shift, FP32 acc}
- **INT4-GPTQ** = {1-level ($g=128$), FP16 dequant, FP32 acc}
- **HiFloat4** = {3-level (hierarchical), E6M2+E1+E1, FP32 acc}

Our contribution is the **search space definition and phase-decoupling methodology**: decomposing the bundled pipeline into this search space, evaluating under both prefill and decode regimes, and demonstrating that (a) existing bundled schemes are suboptimal for both phases, and (b) the optimal configuration differs between prefill and decode—validating the need for phase-adaptive quantization.

C. Tradeoffs and Challenges

- **Search space tractability:** Pruning reduces the combinatorial space to ~ 10 – 20 viable configurations, manageable for exhaustive sweep under both regimes.
- **Phase switching cost:** Supporting two quantization modes requires either two sets of quantized weights in memory or runtime requantization. In practice, scale-factor storage overhead is modest relative to the weight tensors themselves.
- **Design-time commitment:** The accelerator’s compute units are fixed at design time. Phase-adaptive quantization requires the datapath to support at least two rescale modes—we show this is feasible with minimal area overhead.

III. PRELIMINARY RESULTS

We have completed a full hardware cost sweep across 4 Llama-style FFN GEMM shapes (Table II), evaluating 4 configurations under both prefill ($M=128$) and decode ($M=1$) regimes using AccelForge auto-mapping on a parameterized 8×8 PE-array accelerator (64 KB GLB, 128 B RF/PE, 45 nm).

Key findings (Figures 2, 3):

TABLE II
HARDWARE COST COMPARISON: NVFP4 W4A4 VS. FP16 BASELINE ON LLAMA FFN GEMMS. “IDEAL W4A4” SHOWS THE THEORETICAL LOWER BOUND (FP4 MATMUL ONLY, ZERO RESCALE OVERHEAD).

Phase	Config	vs. FP16 Baseline		vs. Ideal W4A4	
		Energy	Latency	Energy	Latency
Decode ($M=1$)	W4A4 NVFP4	0.90×	0.59×	2.84×	9.4×
	Ideal W4A4	0.32×	0.06×	1.0×	1.0×
Prefill ($M=128$)	W4A4 NVFP4	1.27×	0.29×	3.9×	1.16×
	Ideal W4A4	0.33×	0.25×	1.0×	1.0×

- 1) **NVFP4 is energy-beneficial for decode but energy-harmful for prefill.** During decode, the bandwidth savings from 4-bit weights outweigh the rescale overhead (0.90× energy). During prefill, the rescale pipeline’s 74% energy share pushes total cost *above* FP16 (1.27×).
- 2) **Rescale overhead dominates the gap to ideal.** The full NVFP4 pipeline costs 2.8–3.9× more energy than the bare FP4 MatMul. This gap is the optimization target for our search space.
- 3) **Latency scaling differs drastically.** In decode, the NVFP4 pipeline adds 9.4× latency overhead vs. ideal (many extra einsums on a small $M=1$ workload). In prefill, latency overhead is only 1.16× because the large MatMul dominates cycles. This confirms the qualitatively different bottleneck structure.

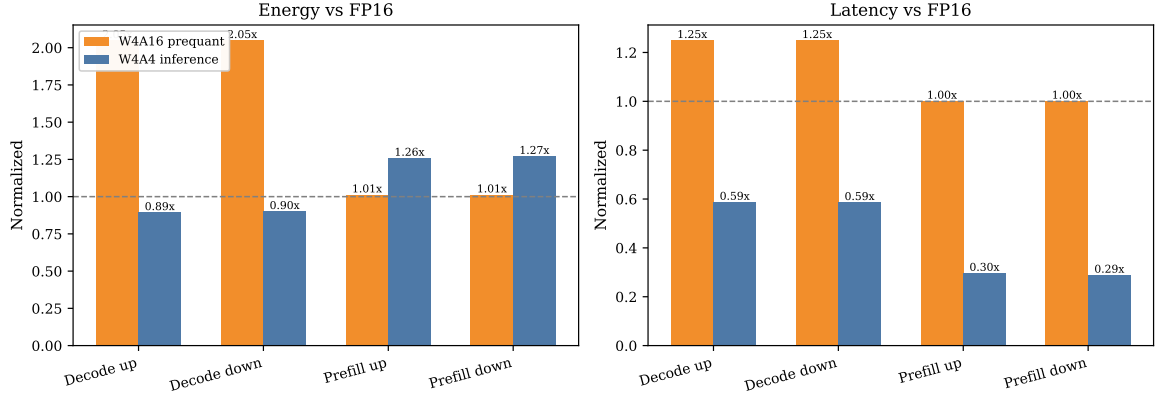


Fig. 2. Energy and latency normalized to FP16 baseline across 4 FFN GEMM shapes. W4A4 NVFP4 improves decode metrics but increases prefill energy by 27%.

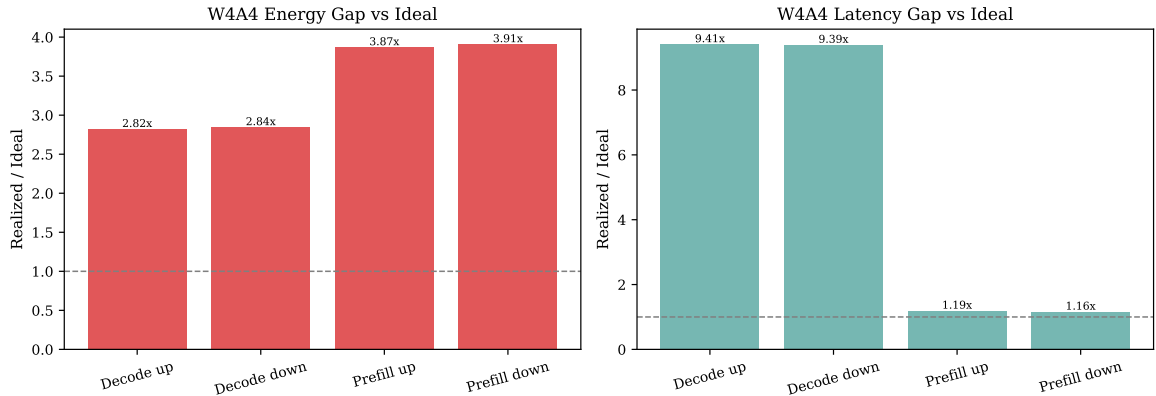


Fig. 3. Realized NVFP4 cost relative to ideal (zero-overhead) W4A4. The 2.8–3.9× energy gap is the optimization headroom targeted by our rescale pipeline DSE.

These results establish two critical facts: (1) the rescale pipeline is the dominant efficiency bottleneck and the natural

target for optimization, and (2) the *same* pipeline configuration behaves oppositely in the two inference phases, motivating phase-specific design.

IV. PROPOSED EXPERIMENTS

A. Baselines, Workloads, and Fair Comparison

We compare against NVFP4 and MXFP4 as baselines—the two dominant hardware-friendly 4-bit formats—evaluated under both prefill and decode regimes. All configurations use 4-bit weights; effective compression ratio (including scale-factor overhead) is reported. M varies to represent prefill ($M = 128$) vs. decode ($M = 1$).

To validate that optimal configurations are *workload-dependent*, we evaluate across **three application domains** with distinct GEMM shape profiles (Table III):

TABLE III
TARGET WORKLOADS SPANNING THREE APPLICATION DOMAINS. EACH HAS DISTINCT GEMM SHAPES AND DEPLOYMENT CONSTRAINTS.

Domain	Representative Model	Key GEMM Shapes	Eval Metric
LLM	Llama-2-7B	FFN: (M, 11008, 4096)	Perplexity
VLM	LLaVA / Qwen-VL	ViT: (M, 3072, 3072)	Task acc.
VLA	OpenVLA / RT-2	Action head: (M, 256, 4096)	Success rate

These domains differ in their GEMM dimension profiles: LLM FFN layers have large N and K ; VLM vision encoders have moderate, balanced dimensions; VLA action heads have small N with large K . Different (N, K) profiles lead to different compute-vs-memory balance, and thus different optimal rescale configurations.

Accuracy metrics: We use two levels of evaluation. (1) *Per-layer*: cosine similarity between quantized and FP16 layer outputs ($\cos_sim = \frac{\langle Y_q, Y_{16} \rangle}{\|Y_q\| \cdot \|Y_{16}\|}$), which captures numerical fidelity without requiring end-to-end inference. (2) *End-to-end* (when feasible): perplexity for LLM, task accuracy for VLM, success rate for VLA.

B. Experiment 1: Phase-Dependent Energy Bottleneck Analysis (Done)

Goal: Demonstrate that prefill and decode have qualitatively different energy bottleneck structures.

Status: Completed. Our preliminary results (Section III, Figures 1–3) confirm: rescale accounts for 74% of prefill energy vs. 64.5% of decode energy. NVFP4 costs $1.27\times$ FP16 in prefill energy but only $0.90\times$ in decode. This asymmetry directly motivates phase-specific search.

Class concepts: Compute-bound vs. memory-bound regimes; energy cost hierarchy; arithmetic intensity.

C. Experiment 2: Cross-Workload Search Space Sweep

Goal: For each of the three workloads, sweep the search space under both prefill and decode, and show that (a) the Pareto-optimal configuration differs between phases within each workload, and (b) the optimal configuration also differs *across* workloads.

Method:

- 1) *Hardware cost*: For each candidate configuration $\times$ workload $\times$ phase, run AccelForge evaluation with the workload’s representative GEMM shapes. Extract energy per output element and latency.
- 2) *Accuracy*: For each configuration, run PyTorch emulation on representative weight matrices from the target model. Measure per-layer cosine similarity; for a subset, measure end-to-end task metric.
- 3) *Joint Pareto analysis*: For each (workload, phase) pair, plot all configurations on the accuracy–energy plane. Draw Pareto frontiers; mark NVFP4 and MXFP4 as reference points.

Class concepts: Design-space exploration; Pareto optimality; phase-aware hardware-software co-design.

Metrics & visualization: A grid of Pareto plots: 3 workloads $\times$ 2 phases = 6 panels. Each panel shows all search-space configurations as scatter points, with the Pareto frontier highlighted and NVFP4/MXFP4 marked. The key visual is that (a) the frontier shape and knee-point differ across panels, and (b) NVFP4/MXFP4 are interior to all frontiers.

Expected trends:

- **LLM** (large K): Prefill frontier favors cheaper rescale (FP16/bit-shift) to cut the dominant compute energy. Decode frontier favors coarser granularity to reduce data movement.
- **VLM** (balanced N, K): Vision encoder GEMMs have moderate dimensions—may tolerate more aggressive quantization (single-level, E8M0) across both phases.
- **VLA** (small N , latency-critical): Action-head GEMMs are small and latency-sensitive. May favor minimal pipeline depth (0 or 1 rescale level) to reduce per-inference overhead.
- **Cross-workload divergence**: The Pareto-optimal configuration for LLM prefill (e.g., 2-level with FP16 rescale) differs from VLA decode (e.g., 1-level with E8M0 shift), confirming that quantization design should be workload-specific.

D. Experiment 3: Workload-Adaptive Gain Quantification

Goal: Quantify the end-to-end efficiency gain of workload-and-phase-adaptive quantization over a single fixed scheme.

Method: For each workload, compute the efficiency of three strategies:

- *Fixed-NVFP4*: Use NVFP4 for both phases (status quo).
- *Best-fixed*: Use the single best configuration from the search space for both phases.
- *Phase-adaptive*: Use separate Pareto-optimal configurations for prefill and decode.

Weighted energy: $E = \alpha \cdot E_{\text{prefill}} + (1 - \alpha) \cdot E_{\text{decode}}$, where α depends on deployment scenario (e.g., $\alpha=0.3$ for chatbot, $\alpha=0.8$ for batch processing, $\alpha=0.5$ for VLA real-time control).

Class concepts: Workload characterization; design-time specialization; system-level optimization.

Metrics & visualization:

- Table: best configuration for each (workload, phase) pair, with energy breakdown and accuracy (cos_sim). Show that the 6 cells contain ≥ 3 distinct configurations.
- Bar chart: energy savings of phase-adaptive vs. fixed-NVFP4 and vs. best-fixed, per workload.
- Line plot: $X = \alpha$, $Y = \text{energy savings}$, one curve per workload. Shows that phase-adaptive wins across all deployment ratios.

Expected outcome: Phase-adaptive quantization yields 15–30% energy reduction over NVFP4 across all three workloads. The gain is largest for LLM prefill (where rescale overhead is highest) and for VLA (where latency constraints favor aggressive pipeline simplification). Crucially, no single configuration from the search space dominates across all 6 (workload, phase) cells—**validating that the search space and per-workload search methodology are both necessary**.

V. TIMELINE AND WORK DISTRIBUTION

Day	Tasks	Status
4.13–4.16	AccelForge baseline sweep (LLM shapes, 4 configs)	Done
	Phase bottleneck analysis (Experiment 1)	Done
4.17–4.21	Extend to VLM/VLA shapes; full search space sweep	In progress
	PyTorch quantization emulator + cos_sim evaluation	In progress
4.21–4.28	Cross-workload Pareto analysis (Experiment 2)	Planned
	Workload-adaptive gain quantification (Experiment 3)	Planned
4.28–5.1	Final figures; write-up and presentation	Planned

Work split: One member focuses on AccelForge hardware modeling (workload/arch YAML generation, mapping evaluation, energy/latency extraction across LLM/VLM/VLA shapes). The other focuses on PyTorch accuracy evaluation (quantization emulator, per-layer cosine similarity, end-to-end metrics). Both collaborate on Pareto analysis and writing.

Progress: Milestones 1–2 (baseline modeling + inference-faithful LLM sweep with prefill/decode comparison) are complete. The hardware evaluation pipeline—automated workload generation, AccelForge auto-mapping, per-einsum breakdown extraction—is fully operational and ready to scale to VLM/VLA workloads and the full search space.

REFERENCES

- [1] Y. Sun et al., “HiFloat4 Format for Language Model Inference,” arXiv:2602.11287, 2026.
- [2] A. Saha et al., “AXE: Accumulator-Aware Post-Training Quantization,” arXiv:2409.17092, 2024.
- [3] Z. Zhang et al., “MixPE: Quantization and Hardware Co-design,” arXiv:2411.16158, 2024.
- [4] T. Jeong et al., “Is Finer Better? Limits of Microscaling Data Formats,” arXiv:2601.19026, 2026.
- [5] J. Park et al., “M²XFP: Metadata-Augmented Microscaling,” ASPLOS, 2026.
- [6] S. Kim et al., “MX+: Pushing the Limits of Microscaling Formats,” MICRO, 2025.